

AI Code Review & Security Analysis Report

Professional Code Quality & Security Assessment

Generated: 2026-08-14 09:54 UTC | Language: python | Model: llama-3.3-70b-versatile

Executive Summary

Code Health Score	Risk Score	Total Findings
0/100	41	12

Merge Readiness: **Not ready for merge due to high-risk security vulnerabilities.**

The code review has identified 12 findings with a risk score of 41. The majority of the findings are critical, indicating a high risk of security vulnerabilities.

Severity Breakdown

Critical	High	Medium	Low
7	4	0	1

Detailed Findings

Type	Severity	Line	Message	Recommendation
1. Hardcoded Password	● High	1	password = "admin123"	Store passwords in environment variables.
2. Hardcoded API Key	● High	3	api_key = "123456789"	Store secrets securely.
3. SQL Injection	● Critical	7	query = "SELECT * FROM users WHERE name=" + name + ""	Use parameterized queries.
4. Dangerous Dynamic Execution	● Critical	9	eval(input())	Avoid dynamic code execution.
5. Dangerous Dynamic Execution	● Critical	11	exec("print('Hello')")	Avoid dynamic code execution.
6. Command Injection	● Critical	15	subprocess.run("dir", shell=True)	Do not use shell=True with user input.

Type	Severity	Line	Message	Recommendation
7. Security	● Critical	1	Hardcoded sensitive information	Use environment variables or a secure secrets management system to store sensitive information like passwords and API keys.
8. Security	● Critical	5	SQL injection vulnerability	Use parameterized queries or prepared statements to prevent SQL injection attacks.
9. Security	● Critical	7	Use of eval() function	Avoid using eval() and instead use safer alternatives like ast.literal_eval() or json.loads() for parsing input data.
10. Security	● High	9	Use of exec() function	Avoid using exec() and instead use safer alternatives like subprocess.run() with a list of arguments to prevent shell injection attacks.
11. Security	● High	12	Use of shell=True in subprocess.run()	Avoid using shell=True and instead pass a list of arguments to subprocess.run() to prevent shell injection attacks.
12. Best Practice	● Low	3	Input validation	Validate and sanitize user input to prevent unexpected behavior or errors.

■ Remediation Roadmap

Priority-based action items to improve code quality and security

Prior ity	Finding Type	Severity	Impact	Suggested Action
#1	SQL Injection	● Critical	High risk of security vulnerability	Use parameterized queries to prevent SQL injection attacks.
#2	Dangerous Dynamic Execution	● Critical	High risk of code injection attacks	Replace eval() and exec() with safer alternatives.
#3	Command Injection	● Critical	High risk of security vulnerability	Avoid using shell=True with user input and pass commands as a list of strings.
#4	Security	● Critical	High risk of security vulnerability	Use environment variables or a secure secrets management system to store sensitive information.
#5	Hardcoded Password	● High	High risk of security vulnerability	Store passwords in environment variables.
#6	Hardcoded API Key	● High	High risk of security vulnerability	Store secrets securely using environment variables or a secrets manager.
#7	Security	● High	High risk of security vulnerability	Avoid using eval() and exec() and instead use safer alternatives.
#8	Security	● High	High risk of security vulnerability	Use parameterized queries to prevent SQL injection attacks.
#9	Security	● High	High risk of security vulnerability	Avoid using shell=True and instead pass a list of arguments to subprocess.run().
#10	Best Practice	● Low	Low risk of security vulnerability	Validate and sanitize user input to prevent unexpected behavior or errors.

■ Key Recommendations

1. Address all Critical and High severity findings before merging.
2. Run additional security scanning tools to complement this analysis.
3. Consider implementing automated testing to catch similar issues in the future.

■ Code Quality Metrics

Overall Health Score: 0/100

Risk Level: Unknown

Maintainability: N/A

Security Score: N/A

■ Source Code Snapshot

```
password = "admin123"
```

```
api_key = "123456789"
```

```
name = input()
```

```
query = "SELECT * FROM users WHERE name='" + name + "'"
```

```
eval(input())
```

```
exec("print('Hello')")
```

```
import subprocess
```

```
subprocess.run("dir", shell=True)
```



This report was generated by the AI Code Review & Security Analysis Agent. Review all findings carefully before implementing changes.

Report generated on 2026-08-14 09:54 UTC